Міністерство освіти і наук України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп’ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №5
Варіант 1
На тему
“Ліниві обчислення: генератори та `itertools`”

Виконав:
Студент групи ФЕП-21
Бардак Д. Т

Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти **generator expressions**, yield/yield from, ітератори та лінивість;
- уміти працювати з **нескінченними послідовностями**;
- освоїти базові будівельні блоки з itertools: islice, takewhile, dropwhile, accumulate, count, cycle, repeat;
- реалізувати: **нескінченний стрім Фібоначчі**, **sliding window**, **take/drop/accumulate**-практики.

Теоретичні відомості

- **Ітератори vs ітеровані об’єкти**
- **Ітероване**: те, по чому можна ітерувати (list, tuple, range, генератор).
- **Ітератор**: об’єкт зі станом, який на кожному кроці повертає наступне значення і зупиняється з StopIteration.
- **Лінивість**: значення обчислюються **по мірі запиту** - це економить пам’ять і дозволяє працювати з нескінченними джерелами.

Хід роботи

У ході виконання лабораторної роботи було досліджено принципи лінивих обчислень та використання генераторів у мові Python на прикладі обробки нескінченної послідовності чисел Фібоначчі. У базовому модулі розроблено функцію-генератор fib_stream(), яка за допомогою інструкції yield нескінченно генерує значення без збереження попередніх станів у пам'яті. Для ефективного управління цим потоком реалізовано набір лінивих інструментів, зокрема функції take та drop на основі itertools.islice, а також генератор префіксних сум із використанням itertools.accumulate. Важливим етапом стала розробка універсальної функції sliding_window на базі структури collections.deque(maxlen=k), що дозволило формувати ковзні вікна без передчасної матеріалізації елементів, і на основі якої було реалізовано розрахунок рухомого середнього. Коректність результатів, правильна обробка крайових випадків (наприклад, коли розмір вікна перевищує довжину потоку) та фактична "лінивість" алгоритмів (обчислення лише запитаної кількості елементів) були успішно доведені автоматизованими тестами у фреймворку pytest. Загальну надійність, чистоту та строгу типізацію створеного коду підтверджено успішним проходженням перевірок лінтерами mypy, black та ruff.



```python
from collections import deque
from collections.abc import Iterable, Iterator
from itertools import accumulate, islice
from typing import TypeVar

T = TypeVar("T")


def fib_stream() -> Iterator[int]:
    """Нескінченний генератор чисел Фібоначчі."""
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b


def take(n: int, it: Iterable[T]) -> list[T]:
    """Матеріалізує перші n елементів ітератора у список."""
    return list(islice(it, n))


def drop(n: int, it: Iterable[T]) -> Iterator[T]:
    """Пропускає перші n елементів ітератора."""
    it_iter = iter(it)
    return islice(it_iter, n, None)


def sliding_window(it: Iterable[int], k: int) -> Iterator[tuple[int, ...]]:
    """Формує ковзне вікно розміру k з ітератора (ліниво)."""
    if k <= 0:
        return

    it_iter = iter(it)
    dq: deque[int] = deque(maxlen=k)

    for _ in range(k):
        try:
            dq.append(next(it_iter))
        except StopIteration:
            return  # Якщо елементів менше за k, нічого не віддаємо

    yield tuple(dq)

    for x in it_iter:
        dq.append(x)
        yield tuple(dq)


def moving_average(it: Iterable[int], k: int) -> Iterator[float]:
    """Обчислює рухоме середнє для вікон розміру k."""
    for window in sliding_window(it, k):
        yield sum(window) / k


def cumulative_sums(it: Iterable[int]) -> Iterator[int]:
    """Обчислює префіксні суми потоку (через accumulate)."""
    yield from accumulate(it)
```

Рис. 1-3 Файл streams.py

```python
from streams import (
    cumulative_sums,
    drop,
    fib_stream,
    moving_average,
    sliding_window,
    take,
)


def main() -> None:
    print("Демонстрація лінивих обчислень (Варіант 1)")

    print("Пропустити перші 5 чисел Фібоначчі і взяти 10:")
    stream = fib_stream()
    dropped = drop(5, stream)
    result_1 = take(10, dropped)
    print(f"Результат: {result_1}\n")

    print("Ковзні вікна (k=3) для наступних 5 чисел Фібоначчі:")
    stream2 = fib_stream()
    windows = sliding_window(stream2, 3)
    result_2 = take(5, windows)
    for w in result_2:
        print(f"  {w}")
    print()

    print("Рухоме середнє (k=3) для наступних 5 чисел Фібоначчі:")
    stream3 = fib_stream()
    m_avg = moving_average(stream3, 3)
    result_3 = take(5, m_avg)
    for avg in result_3:
        print(f"  {avg:.2f}")
    print()

    print("Префіксні суми перших 8 чисел Фібоначчі:")
    stream4 = fib_stream()
    sums = cumulative_sums(stream4)
    result_4 = take(8, sums)
    print(f"Результат: {result_4}")


if __name__ == "__main__":
    main()
```

Рис. 4-5 Файл main.py

```python
import os
import sys

sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from collections.abc import Iterator

from streams import (
    cumulative_sums,
    drop,
    fib_stream,
    moving_average,
    sliding_window,
    take,
)


def test_fib_stream() -> None:
    """Перевірка перших значень стріму Фібоначчі."""
    assert take(10, fib_stream()) == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]


def test_take_drop() -> None:
    """Перевірка роботи конвеєра drop + take."""
    stream = fib_stream()
    dropped = drop(5, stream)
    assert take(3, dropped) == [5, 8, 13]


def test_sliding_window_basic() -> None:
    """Перевірка формування вікон на простому списку."""
    xs = [1, 2, 3, 4, 5]
    windows = list(sliding_window(xs, 3))
    assert windows == [(1, 2, 3), (2, 3, 4), (3, 4, 5)]


def test_sliding_window_edge_cases() -> None:
    """Перевірка граничних випадків для вікон."""

    assert list(sliding_window([], 3)) == []

    assert list(sliding_window([1, 2], 3)) == []

    assert list(sliding_window([1, 2, 3], 0)) == []
    assert list(sliding_window([1, 2, 3], -1)) == []


def test_moving_average() -> None:
    """Перевірка обчислення рухомого середнього."""
    xs = [2, 4, 6, 8]

    assert list(moving_average(xs, 2)) == [3.0, 5.0, 7.0]


def test_cumulative_sums() -> None:
```

```python
    """Перевірка префіксних сум."""
    assert list(cumulative_sums([1, 2, 3, 4])) == [1, 3, 6, 10]


def test_laziness() -> None:
    """Тест на підтвердження лінивості (не матеріалізує зайве)."""
    counter = 0

    def mock_stream() -> Iterator[int]:
        nonlocal counter
        while True:
            counter += 1
            yield counter

    stream = mock_stream()

    take(5, stream)
    assert counter == 5

    stream2 = mock_stream()
    counter = 0
    list(take(2, sliding_window(stream2, 3)))
    assert counter == 4
```

Рис. 6-8 Файл test_streams.py

```
(.venv) arch-laptop% mypy .
Success: no issues found in 3 source files

(.venv) arch-laptop% black .
All done! ✨ 🍰 ✨
3 files left unchanged.

(.venv) arch-laptop% ruff check .
All checks passed!

(.venv) arch-laptop%
```

```
(.venv) arch-laptop% pytest .
============================= test session starts ==============================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/ritokk/Documents/fp/lab05/lab05
plugins: platformdirs-4.12.4
collected 7 items

tests/test_streams.py .......                                            [100%]

============================== 7 passed in 0.05s ===============================
(.venv) arch-laptop%
```

Рис.9-10 Успішно виконані тести

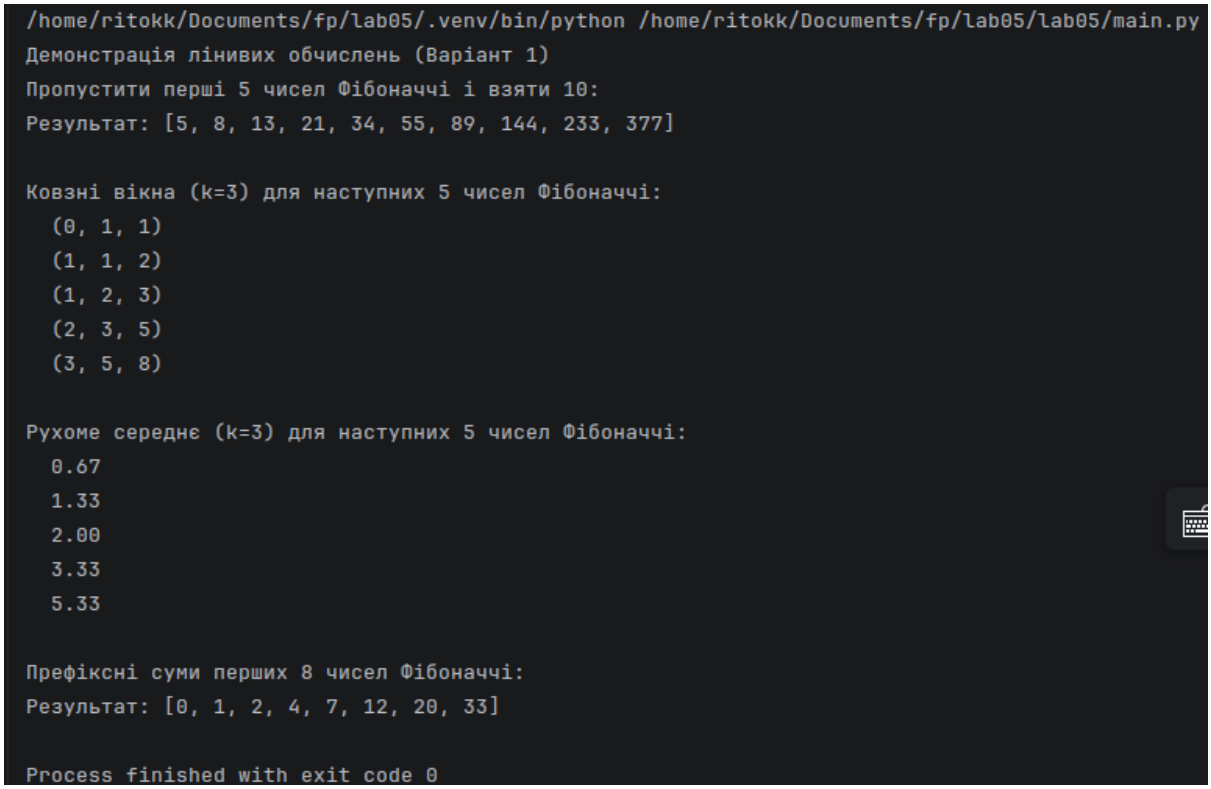


Рис. 11 Запуск програми

Висновок:

Під час виконання лабораторної роботи було практично засвоєно парадигму лінивих обчислень та механізми роботи ітераторів у Python. Побудова конвеєра для нескінченного потоку чисел Фібоначчі за допомогою власних генераторів (yield) та інструментів itertools (зокрема islice та accumulate) довела, що такий підхід дозволяє обробляти об'ємні або нескінченні дані з мінімальними витратами пам'яті. Реалізація лінивого "ковзного вікна" (sliding_window) та префіксних сум підтвердила ефективність потокової обробки без передчасної матеріалізації в списки. Автоматизоване тестування успішно підтвердило не лише коректність математичних розрахунків і обробку крайових випадків, але й сам факт "лінивості" виконання, що є критично важливим для надійної розробки функціональних пайплайнів.